



7ª edición

Ingeniería del Software

Ian Sommerville

PEARSON
Addison
Wesley



Prólogo

La primera edición de este libro de ingeniería del software fue publicada hace más de veinte años. Aquella edición fue escrita utilizando un terminal de texto conectado a una minicomputadora (un PDP-11) que posiblemente costaba cerca de 50.000 \$. Yo he escrito esta edición desde un portátil con conexión inalámbrica que cuesta menos de 2.000 \$ y mucho más potente que aquel PDP-11. El software más común era el software para mainframes, pero las computadoras personales estaban a punto de aparecer. Ninguno de nosotros imaginó el nivel de difusión que éstas iban a tener ni el cambio que este hecho iba a producir en el mundo.

Los cambios en el hardware en los últimos veinte años han sido notables, y podría parecer que los cambios en el software han sido igual de significativos. Ciertamente, nuestra capacidad para construir sistemas grandes y complejos ha mejorado drásticamente. Nuestros servicios e infraestructuras nacionales —energía, comunicaciones y transporte— dependen de sistemas informáticos muy grandes, complejos y fiables. En la construcción de sistemas software se mezclan muchas tecnologías —J2EE, .NET, EJB, SAP, BP4WS, SOAP, CBSE— que permiten que aplicaciones grandes basadas en Web sean desarrolladas mucho más rápido que en el pasado.

Sin embargo, a pesar de los cambios que ha habido en las dos últimas décadas, cuando nosotros miramos más allá de las tecnologías, hacia los procesos fundamentales de la ingeniería del software, éstos se han mantenido igual. Nosotros reconocimos hace veinte años que el modelo en cascada tenía problemas serios, pero un examen publicado en diciembre de 2003 por *IEEE* mostraba que más de un 40% de las compañías siguen utilizando esta aproximación. El teste sigue siendo la técnica de validación dominante, a pesar de que otras técnicas, como las inspecciones, han sido utilizadas de una forma más efectiva desde mediados de los años 70. Las herramientas CASE, a pesar de estar basadas ahora en UML, siguen siendo básicamente editores gráficos con alguna funcionalidad para chequear y generar código.

Nuestros actuales métodos y técnicas de ingeniería del software han hecho que la construcción de sistemas grandes y complejos sea mejor. A pesar de ello, sigue siendo habitual encontrar proyectos que se retrasan, que sobrepasan el presupuesto o que se entregan sin satisfacer las necesidades de los clientes. Mientras estaba escribiendo este libro, se divulgó una investigación del gobierno en Reino Unido, sobre un proyecto para proveer a los juzgados de un sistema software para casos de delincuencia menor. El coste del sistema fue estimado en 156 millones de libras y fue planificado para ser entregado en el año 2001. En 2004, el coste había subido a 390 millones de libras y no estaba totalmente operativo. Hay, por lo tanto, una necesidad imperiosa de educación en ingeniería del software.

En los últimos años, el desarrollo más significativo en ingeniería del software ha sido la aparición de UML como estándar para la descripción de sistemas orientados a objetos, y el desarrollo de métodos ágiles como la programación extrema. Los métodos ágiles están permitiendo el desarrollo rápido de sistemas, explícitamente implican al usuario en el equipo de trabajo y reducen el papeleo y la burocracia en el proceso software. A pesar de lo que algunos críticos sostienen, pienso que estas aproximaciones encarnan buenas prácticas de ingeniería del software. Ellas tienen unos procesos bien definidos, prestan atención a la especificación del sistema y a los requerimientos del usuario, y tienen estándares de alta calidad.

No obstante, esta revisión no pretende ser un texto sobre métodos ágiles. Prefiero centrarme en los procesos básicos de ingeniería del software —especificación, diseño, implementación, verificación, y validación y gestión—. Es necesario entender estos procesos y las técnicas asociadas para decidir si los métodos ágiles son la estrategia de desarrollo más adecuada y cómo adaptar los métodos a una situación particular. Un tema dominante en el libro son los sistemas críticos —sistemas en los que los fallos de funcionamiento tienen consecuencias nefastas y donde la seguridad del sistema es crítica. En cada parte del libro, estudiaremos las técnicas específicas de ingeniería del software que son relevantes para la construcción sistemas críticos.

Inevitablemente, los libros reflejan las opiniones y prejuicios de sus autores. Algunos lectores estarán en desacuerdo con mis opiniones y con mi elección del material. Este desacuerdo es un reflejo de la diversidad de disciplinas y es esencial para su evolución. No obstante, yo espero que a todos los ingenieros de software y estudiantes de ingeniería del software les resulte interesante.

Estructura del libro

La estructura del libro está basada en los procesos fundamentales de la ingeniería del software. Está organizado en seis partes, con varios capítulos en cada parte:

Parte 1: Introduce la ingeniería del software, situándola en un amplio contexto de sistemas y presentando las nociones de procesos y gestión de ingeniería del software.

Parte 2: Trata los procesos, técnicas y documentación asociados con los requerimientos de ingeniería. Incluye un estudio sobre los requerimientos software, modelado de sistemas, especificación formal y técnicas para especificar la fiabilidad.

Parte 3: Esta parte está dedicada al diseño de software y a los procesos de diseño. Tres de los seis capítulos se centran en el importante tema de las arquitecturas software. Otros temas incluyen diseño orientado a objetos, diseño de sistemas en tiempo real y diseño de interfaces de usuario.